Department of Information Technology

"IoT Based Smart Agriculture Monitoring System"

Submitted in partial fulfillment of the requirements

of the degree of

Bachelor of Engineering

By

07 Harshal Bhosale

13 Rakesh Chaudhary

15 Pranit Chiman

73 Om Darekar

Supervisor:

(Prof. Janhavi Baikerikar)



The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

A.Y. 2026-2027

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled "IoT Based Smart Agriculture Monitoring System" is a bonafide work of

07	Harshal Bhosale
13	Rakesh Chaudhary
15	Pranit Chiman
73	Om Darekar

submitted to the University of Mumbai in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology.

Date: /11/2026

(Prof. Janhavi Baikerikar)
Supervisor

(Dr. Sunantha G.)
HOD, IT Department

(Dr. Sudhakar Mande)
Principal

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

Department of Information Technology

Project Report Approval for B.E.

This project report entitled "IoT Based Smart Agriculture Monitoring System" by Harshal Bhosale, Rakesh Chaudhary, Pranit Chiman and Om Darekar is approved for the degree of Bachelor of Engineering in Information Technology.

1. _____

2. _____

(Examiner's Name and Signature)

_____

(Prof. Janhavi Baikerikar)
Supervisor

Date: /11/2026

Place: Kurla (West)

The Bombay Salesian Society's

Don Bosco Institute of Technology, Mumbai

(An Autonomous Institute Affiliated to the University of Mumbai)

Department of Information Technology

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact or source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of the Student	**Roll No.**	**Signature**
Harshal Bhosale	07	
Rakesh Chaudhary	13	
Pranit Chiman	15	
Om Darekar	73	

Date: /11/2026

Abstract

Agriculture relies on timely knowledge of soil and environmental conditions. Manual field inspection can delay recognition of dry soil, excessive moisture, high temperature, unfavorable humidity and changes in local gas-sensor readings. A low-cost monitoring interface can help farmers assess conditions and plan timely actions.

This report describes an Internet of Everything (IOE) based Smart Agriculture Monitoring System built around an ESP32 development board. A DHT11 measures ambient temperature and humidity; a resistive soil probe provides an analog wetness reading; and an MQ7 module contributes a raw gas-sensor signal. The ESP32 samples these sensors and makes measurements available through its built-in Wi-Fi connection.

An ESP32 HTTP web server responds with JSON at /data. A standalone browser dashboard implemented in HTML, CSS and JavaScript polls this endpoint, displays live readings, status indicators and short trend graphs, and evaluates configurable alert thresholds. The browser independently retrieves the next 12 hours of forecast information from OpenWeatherMap using a user-entered key.

The decision process converts calibrated raw soil input into an indicative moisture percentage, distinguishes dry soil from possible waterlogging, and issues heat, humidity and raw-gas alerts. If rain is likely, the dashboard may recommend delaying irrigation despite dry soil. The MQ7 readings are not calibrated CO concentrations and the prototype does not operate irrigation pumps or other actuators.

The supplied firmware, dashboard and working hardware demonstrate a local-network agricultural monitoring and advisory system linking Things, Data, Processes and People. Future improvements focus on validated sensing, secure connectivity, history retention and interface reliability, while preserving human control over agricultural actions.

Keywords: Internet of Everything, Smart Agriculture, ESP32, DHT11, MQ7, Soil Moisture, HTTP, OpenWeatherMap.

Contents

1 Introduction 1
1.1 Problem Statement 1
1.2 Scope of the Project 1
1.3 Current Scenario 2
1.4 Need for the Proposed System 2
1.5 Summary of the Results and Tasks Completed 3
2 Review of Literature 4
2.1 Summary of the Investigation in Published Papers and Technical Sources 4
2.2 Comparison Between Tools / Methods / Algorithms 5
2.3 Algorithm(s) with Example 6
3 Analysis and Design 7
3.1 Methodology / Procedure Adopted 7
3.2 Analysis 7
3.2.1 Software / System Requirement Specification – IEEE Format 8
3.3 System Architecture / Design 8
3.3.1 Modules and Their Description 9
4 Implementation 13
4.1 Implementation Plan for Sem – 8 13
4.2 Coding Standard 14
4.3 Testing 14
4.3.1 Test Cases 14
4.3.2 Results of Testing and System Performance 15
5 Results and Discussion 16
6 Conclusion & Future Work 18
Appendix - I: Data Sheets and Installation Procedure 19
Appendix - II: Software Requirement Specification 21
Appendix - III: Source Code Listings 23
References 26
Acknowledgements 27
Sustainable Development Goals 28
Technical Paper in IEEE Format 29

List of Figures

Figure 3.1: Block diagram of the IoE Smart Agriculture architecture 8

Figure 3.2: Circuit diagram showing ESP32 and sensor wiring 9

Figure 3.3: Flowchart of sensing and decision-support process 11

Figure 3.4: Data Flow Diagram – Level 0 11

Figure 3.5: Data Flow Diagram – Level 1 12

Figure 4.1: Proposed Semester VIII implementation schedule 13

Figure 5.1: Complete Smart Agriculture hardware setup 16

Figure 5.2: ESP32 and sensor connection close-up 16

Figure 5.3: Web dashboard with active warnings 17

Figure 5.4: Web dashboard and calibration settings 17

List of Tables

Table 1.1: Modules planned and achieved 3
Table 2.1: Summary of reviewed project sources 4
Table 2.2: Comparison of agricultural monitoring methods 5
Table 2.3: Comparison of local vs cloud architecture 5
Table 2.4: Time and space complexity of project algorithms 6
Table 3.1: Requirements refined during development 7
Table 3.2: Cost analysis of the sensor endpoint 7
Table 3.3: The four pillars of IOE in the project 8
Table 3.4: Technology stack 8
Table 3.5: JSON schema of the ESP32 /data endpoint 8
Table 3.6: Comparison with traditional field monitoring 9
Table 3.7: GPIO pin mapping 9
Table 4.1: Work Breakdown Structure for Sem – 8 13
Table 4.2: Individual contribution 13
Table 4.3: Naming conventions used 14
Table 4.4: Functional test cases 14
Table 4.5: Results of functional testing 15
Table 5.1: Comparison between proposed system and conventional practice 17
Table A.1: ESP32 development board 19
Table A.2: DHT11 sensor 19
Table A.3: Soil moisture sensor 19
Table A.4: MQ7 module 19

1 Introduction

1.1 Problem Statement

Agricultural management depends on frequent assessment of soil moisture and ambient conditions. Manual checks can be infrequent, and their observations are not continuously available to the farmer. Soil may become too dry or overwatered while changes in temperature, humidity or gas-sensor readings go unnoticed.

Existing monitoring solutions may depend on a proprietary mobile application or a remote platform, restricting inspection of live readings without an account or external infrastructure. Irrigation decisions based only on current dryness may also waste water when rainfall is likely.

Aim of the project: to implement a low-cost Internet of Everything (IOE) based Smart Agriculture Monitoring System that links ESP32 sensor readings to a locally accessible custom browser dashboard, integrates an external weather forecast, and provides understandable, rule-based recommendations. All irrigation and ventilation actions remain the responsibility of the user.

1.2 Scope of the Project

The project scope and assumptions are as follows:

- **Live environmental monitoring:** temperature and relative humidity from DHT11, estimated moisture from a resistive soil probe, and a raw MQ7 analog signal.
- **Local connectivity:** ESP32 Wi-Fi station mode with HTTP data access at /data and browser polling, without Blynk or a project cloud database.
- **Custom web application:** responsive sensor cards, trend indicators, connectivity status, calibration controls and practical recommendations.
- **Weather-aware advice:** OpenWeatherMap forecast data considered together with local readings for irrigation decisions.
- **User-controlled action:** the dashboard informs farmers; it does not switch irrigation pumps, fans or other actuators.

Assumptions and limits: the ESP32 and browser can reach each other through the local network; the browser has internet access when forecast data is used; the user supplies the OpenWeatherMap API key. Soil moisture percentage is an indicative calibrated index, and the MQ7 signal is not reported as certified CO ppm.

- **Requirements gathering phase:** identify sensor parameters, local-network requirements and crop-related decision needs.
- **Requirements analysis phase:** select ESP32 and three existing sensors, determine the dashboard JSON contract, assess calibration and safety limitations.
- **System design phase:** develop local server architecture, dashboard panels, weather call and threshold rules.
- **Validation phase:** check hardware readings, browser access and working forecast integration on the actual prototype.

1.3 Current Scenario

Manual field inspection provides measurements only when a farmer visits the site. Readings may be recorded informally and may not be immediately comparable against practical thresholds. A stand-alone sensor display also cannot combine local field measurements with an upcoming rain forecast.

- Soil irrigation can be mistimed because the soil is inspected too rarely.
- Changes in local temperature and humidity may not be apparent until the next inspection.
- A raw gas-sensor response may be difficult to interpret without a clear indication that it is uncalibrated.
- A proprietary monitoring application may introduce unnecessary accounts or platform dependence.

1.4 Need for the Proposed System

An accessible local monitoring system is needed to present field conditions and prompt informed action without depending on automatic control. The selected architecture satisfies this need by:

- making live measurements available on an ordinary browser through the ESP32 IP address;
- transforming a resistive probe reading into an estimated percentage with adjustable wet/dry calibration;
- indicating dry soil, excess moisture, heat, high humidity and elevated raw gas-sensor levels using configurable limits;
- consulting near-term weather conditions to avoid recommending unnecessary irrigation before likely rainfall;
- retaining settings locally and displaying recent on-screen trends;
- connecting People, Process, Data and Things in one low-cost academic prototype.

1.5 Summary of the Results and Tasks Completed

Table 1.1: Modules planned and achieved

Module	Planned work	Status
Sensor node (ESP32)	Read DHT11 and average analog soil/MQ7 data	Working on hardware
Local HTTP service	Expose JSON measurements on /data	Working
Custom browser dashboard	Live values, status, trends and calibration	Working
Weather service	Fetch 12-hour forecast through OpenWeatherMap	Working
Decision-support rules	Dry soil, rain, heat, humidity and gas guidance	Implemented
Persistent logging and secure remote access	Potential later enhancement	Future scope

The hardware, dashboard and forecast display have been operated successfully on the real prototype as confirmed by the project team. No calibrated reference dataset, field water-savings result or independently measured end-to-end latency has been supplied; these results are not claimed.

In the embedded layer, the DHT11 provides two environmental quantities through one digital signal line, while GPIO34 and GPIO35 acquire the independent analog channels. The ESP32 caches these measurements and makes them available through a lightweight web service rather than pushing them to a separate database. This division keeps sensing and networking on the microcontroller, while interpretation remains in the browser.

In the interface, the farmer enters the ESP32 address and starts live polling. The dashboard displays the current readings with short in-session traces and evaluates advice using calibration and warning limits saved locally in the browser. Weather lookup is optional: the forecast is retrieved through OpenWeatherMap, and failure to configure the key does not prevent the ESP32 sensor display from working.

2 Review of Literature

2.1 Summary of the Investigation in Published Papers and Technical Sources

The source report surveys smart agriculture approaches and supplies primary technical references for the parts used in this prototype. The discussion below is restricted to those references rather than introducing unsupported claims about unpublished experiments or measured performance.

1. ESP32 Technical Reference Manual [1]

Describes the ESP32 processor, Wi-Fi and analog-to-digital interface. It supports the choice of the ESP32 for both sensor acquisition and local HTTP service.

2. DHT11 Technical Data Sheet [2]

Documents the sensor's operating range, basic precision limits and single-wire digital data interface. These constraints limit the prototype to general ambient monitoring.

3. MQ7 Manufacturer Information [3]

Describes MQ7 gas-sensing characteristics and the need for appropriate heater operation and calibration. The project therefore displays raw ADC output, not a calibrated CO concentration.

4. Adafruit DHT Sensor Library [4] and Arduino Documentation [9]

Provide software support for reading DHT11 values and building an ESP32 embedded application with Arduino C/C++.

5. OpenWeather Forecast API [5]

Provides the 5-day / 3-hour forecast endpoint used by the browser to obtain the next four forecast samples and evaluate near-term rain probability.

6. Agriculture IoT Review [8]

The cited 2017 review introduces IoT sensing opportunities and challenges for agriculture; this project demonstrates a small monitoring and decision-support implementation rather than proving crop-yield changes.

Table 2.1: Summary of reviewed project sources

Ref.	Area	Project relevance
[1]	ESP32 hardware	Wi-Fi and ADC1-based acquisition
[2]	DHT11	Temperature / humidity operating limits
[3]	MQ7	Gas sensing and calibration caution
[4], [9]	Arduino libraries	Sensor acquisition and serial diagnostics
[5]	Forecast API	Rain-aware decision support
[8]	Agriculture IoT literature	Context for sensing and decision support

2.2 Comparison Between the Tools / Methods / Algorithms

The system was selected for local accessibility, low hardware complexity and direct user control. The comparisons reflect the design alternatives discussed in the original project report.

Compared with the earlier Blynk-based implementation described in the source report, the present interface is a self-contained browser page. It does not require the Blynk service to render sensor readings, although the viewing device must still reach the ESP32 over Wi-Fi. The sensor node and the forecast API serve different roles: the former reports conditions measured at the installed hardware, while the latter supplies broader predictions for the selected city.

The design deliberately limits the meaning of displayed results. A soil percentage is a relative index derived from two user-selected raw readings, not a laboratory measurement of water content. Likewise, the MQ7 card shows an ADC number that can be compared with configurable warning limits, not a calibrated carbon-monoxide concentration. These distinctions are central to interpreting the comparison tables.

Table 2.2: Comparison of agricultural monitoring methods

Parameter	Manual inspection	Blynk-based IoT	Proposed local dashboard
Readings	Occasional field checks	App-based remote values	Browser-based live values
Platform dependency	None	Blynk mobile service	No Blynk account
Forecast-aware advice	Manual judgement	Depends on app configuration	Built-in browser weather rules
Soil calibration	Observation-based	Depends on implementation	Editable dry/wet ADC values
Actuator control	Manual	May be configured	Not implemented; advisory only

Table 2.3: Comparison of local vs cloud architecture

Aspect	ESP32 local HTTP (adopted)	Cloud-dependent alternative
Connectivity	Browser reaches ESP32 on local Wi-Fi	Requires external data platform
Storage	No permanent sensor-history database	May support stored history
Software	Independent HTML/CSS/JavaScript file	Cloud dashboard or app
Remote reach	Not open to unrestricted internet	Requires authenticated architecture
Credentials	Wi-Fi in firmware; weather key in browser	Cloud credential management required

2.3 Algorithm(s) with Example

Algorithm 1: Averaged sensor acquisition on ESP32

```
Every ~2 seconds:
    read DHT11 temperature and humidity
    set dhtOk = true only if both readings are valid
    for each analog sensor (soil and MQ7):
        take 10 ADC readings with brief delay
        return integer average
    retain latest measurements
On GET /data: return JSON of cached measurements
```

Example: a soil input is read ten times and the firmware returns their arithmetic mean as the raw soil field. If the DHT11 read fails, dhtOk becomes false while previous temperature/humidity values remain cached.

Algorithm 2: Soil moisture conversion and rain-aware irrigation advice

```
soilPct = clamp(0, 100, (dryReading - rawSoil) /
                        (dryReading - wetReading) * 100)
if soilPct < soilLow:
    if forecastRainProbability >= 0.60:
        suggest postponing irrigation
    else:
        suggest irrigation
else if soilPct > soilHigh:
    warn of possible waterlogging
```

Example using editable defaults: dry = 3200, wet = 1400 and raw = 2700 produces 27.8% estimated moisture. This is below the default 30% irrigation threshold. If the near-term maximum rain probability is at least 60%, the advice changes to wait rather than irrigate immediately.

Algorithm 3: Warning evaluation

The browser checks configured temperature limits (32°C warning; 38°C critical), raw MQ7 limits (1500 warning; 2500 critical), soil-waterlogging percentage (85%) and combined heat–humidity conditions. The overall banner summarizes the severity of the generated advisories. Thresholds are user adjustable and are not validated crop-specific standards.

Table 2.4: Time and space complexity of project algorithms

Algorithm	Time complexity	Space complexity
Analog averaging (10 samples)	O(10) = O(1) per sensor	O(1)
Soil calibration formula	O(1)	O(1)
Threshold-based recommendations	O(1) per refresh	O(1)
Forecast reduction (four 3h samples)	O(4) = O(1)	O(1)
Trend storage (30 readings per channel)	O(30) per redraw	O(30) per series

3 Analysis and Design

3.1 Methodology / Procedure Adopted

Development proceeded through connected stages: sensor and pin selection, ESP32 wiring, firmware data acquisition, local HTTP endpoint testing, browser interface implementation, OpenWeatherMap integration, calibration controls, decision rules and real-hardware demonstrations. The implementation replaces the previous semester's Blynk application with a dedicated browser dashboard while retaining the essential sensor connections.

Validation used the ESP32 Serial Monitor and the locally opened dashboard to check that all four readings display properly. The weather panel was also confirmed to load successfully. No claimed statistical measurement dataset is available in the provided report and source files.

3.2 Analysis

Technical feasibility: the ESP32 includes 2.4 GHz Wi-Fi, digital GPIO and ADC1 analog inputs, so it can acquire the selected sensors and serve a small JSON response. HTML, CSS and JavaScript run in a standard browser; the OpenWeatherMap API adds forecast information separately.

Economic feasibility: the source report estimates ₹1,280 for the full hardware list, including a separately listed optional supply. Arduino IDE, the ESP32/DHT libraries and a locally opened HTML dashboard require no mandatory application license for this demonstration.

Operational feasibility: the user enters the ESP32 IP, presses Connect, and inspects the live readings and agricultural advisories. Data acquisition stays local and no pump or ventilation equipment is controlled.

Table 3.1: Requirements refined during development

Requirement	Earlier approach	Current approach	Reason
User interface	Blynk mobile app	Custom HTML/JS dashboard	Independent browser UI
Device data	Blynk transmission	Local HTTP /data JSON	Direct local access
Irrigation guidance	Soil observation	Soil + rain forecast	Context-aware advice
Sensor limits	Preselected values	Browser-editable thresholds	Adjustable demonstration
Actuators	Not part of this version	Recommendations only	Human retains control

Table 3.2: Cost analysis of the sensor endpoint

Sr. No.	Component	Qty	Approx. cost (INR)	Use
1	ESP32 development board	1	500	Wi-Fi controller
2	DHT11	1	100	Ambient sensing
3	Soil moisture probe	1	80	Analog soil sensing
4	MQ7 gas sensor	1	150	Raw gas signal
5	Breadboard	1	120	Prototyping
6	Jumper wires	1 set	100	Wiring
7	USB cable	1	80	Power/programming
8	External power supply (optional)	1	150	Standalone power
	Total		1,280	Estimate from own report

3.2.1 Software / System Requirement Specification – IEEE Format

The Software Requirement Specification tailored to this ESP32 monitoring and advisory project appears as Appendix II. It describes the functional and interface requirements, operating assumptions, quality considerations and the exclusion of automatic control.

3.3 System Architecture / Design

The system combines physical sensing, local network exchange, independent weather information and browser-side decision support. The ESP32 acquires readings and publishes JSON; the dashboard performs display, calibration and recommendation logic.

Table 3.3: The four pillars of IOE in the project

Pillar	Components in the implemented project
Things	ESP32 with DHT11, resistive soil-moisture and MQ7 modules
Data	Digital temperature/humidity, analog averages, JSON and forecast values
Process	Polling, calibration conversion, alert thresholds and rain-conditioned advice
People	Farmer or operator who reads recommendations and acts manually

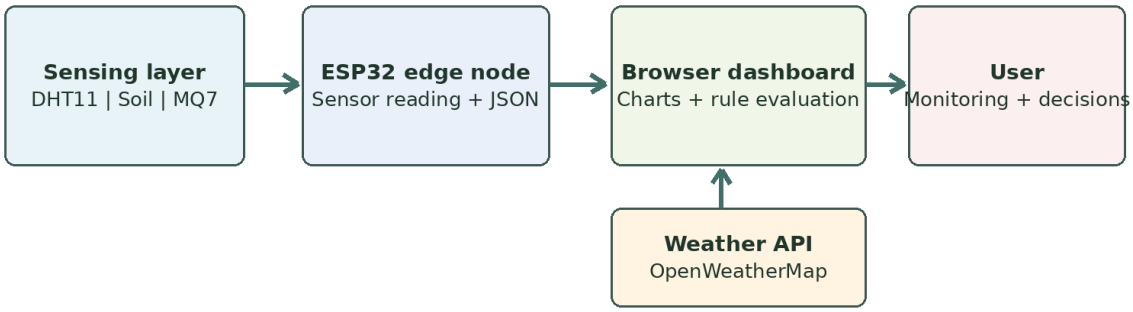


Figure 3.1: Block diagram of the IoE Smart Agriculture architecture

Table 3.4: Technology stack

Technology	Role
ESP32 Arduino C++	Read sensors, manage Wi-Fi and serve JSON
WiFi.h / WebServer.h	Local HTTP networking at port 80
Adafruit DHT library	DHT11 acquisition
HTML, CSS and JavaScript	Interactive dashboard, visualizations and rules
Browser localStorage	Preserve settings and calibration limits
OpenWeatherMap 5-day / 3-hour API	Fetch near-term forecast in browser

Table 3.5: JSON schema of the ESP32 /data endpoint

Field	Type	Meaning
temperature	Number	Last valid DHT11 temperature in °C
humidity	Number	Last valid DHT11 relative humidity in %
soil	Integer	Average raw soil ADC reading
gas	Integer	Average raw MQ7 ADC reading
dhtOk	Boolean	Whether latest DHT11 read succeeded

Table 3.6: Comparison with traditional field monitoring

Feature	Conventional practice	Proposed IOE system
Data visibility	Field visit / manual notes	Periodic local dashboard refresh
Soil decisions	Observed wetness only	Calibration + rain-aware advice
Heat awareness	Occasional observation	Live temperature/humidity warnings
Gas sensing	Often absent	Raw MQ7 signal and caution
Control	Manual	Manual, informed by dashboard

3.3.1 Modules and Their Description

Module 1 – ESP32 sensor node

The ESP32 reads the DHT11 on GPIO4, soil sensor on GPIO34 and MQ7 on GPIO35. Analog inputs are on ADC1-capable pins, avoiding the known ESP32 ADC2/Wi-Fi conflict. All three attached sensors are operated at 3.3 V in the actual assembled prototype, as confirmed by the team.

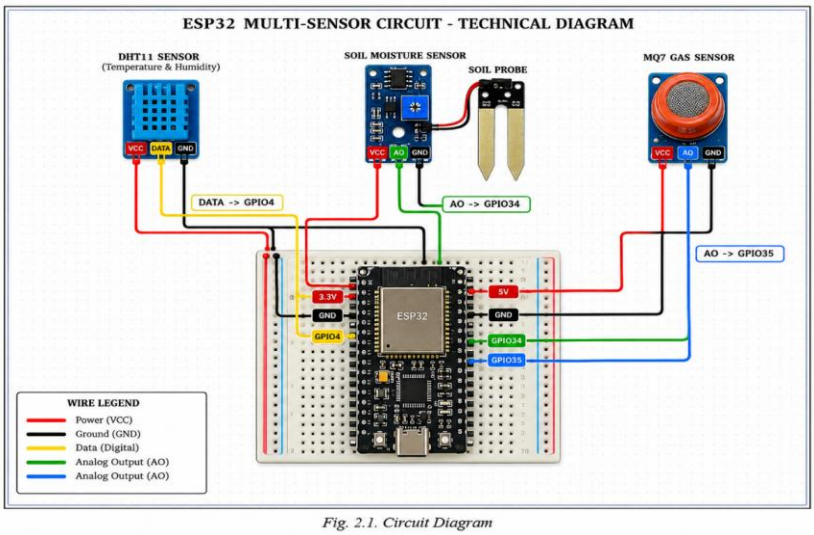


Fig. 2.1. Circuit Diagram

Figure 3.2: Circuit diagram showing ESP32 and sensor wiring

Table 3.7: GPIO pin mapping

Device	ESP32 pin	Function
DHT11 data	GPIO4	Digital temperature/humidity
Soil moisture AO	GPIO34	ADC1 analog input
MQ7 AO	GPIO35	ADC1 analog input

Module 2 – Local HTTP service

The firmware uses WebServer(80), registers the /data endpoint and sends JSON on HTTP GET. An Access-Control-Allow-Origin header supports a separate locally opened browser page; Cache-Control: no-store prevents use of previously cached response data.

Module 3 – Custom dashboard

The browser displays temperature, humidity, estimated soil moisture and raw gas cards with animation and short trends. A connection indicator, global status banner, editable limits and an advice panel help the user interpret measurements. Polling defaults to every three seconds.

Module 4 – Forecast integration

The browser sends HTTPS requests to OpenWeatherMap for a selected city and the user-provided key. The application examines the next four three-hour forecast records and uses the greatest probability of precipitation when offering irrigation advice.

Module 5 – Decision-support rules

JavaScript evaluates dry soil, likely rain, waterlogging, heat, humidity and raw gas warnings against configurable thresholds. No relay, pump, fan or automatic switching mechanism is controlled by these rules.

The decision procedure first converts the resistive probe signal into a bounded percentage using the configured dry and wet readings. Soil below the lower threshold normally produces an irrigation recommendation; when the weather result shows a probability of precipitation of at least 60%, the advice may instead suggest waiting. Soil above the upper threshold produces a possible waterlogging message. These messages support manual decisions and do not command a water pump.

Temperature, humidity and gas conditions are assessed in the same JavaScript function. Heat warnings use two configurable temperature limits, while gas warnings compare the raw MQ7 ADC value against separate warning and critical levels. A combined hot-and-humid condition also provides an advisory message when its rule is reached. The browser assigns an overall status from the most severe recommendation, and each card retains its own visual threshold state.

The ESP32 and the weather service communicate independently with the browser. Thus, unavailable internet weather data need not prevent local sensing, and disconnected Wi-Fi prevents fresh sensor updates even when a previously retrieved forecast can still be displayed. The in-browser charts are short trend views that reset with the session; historical cloud storage is outside the present implementation.

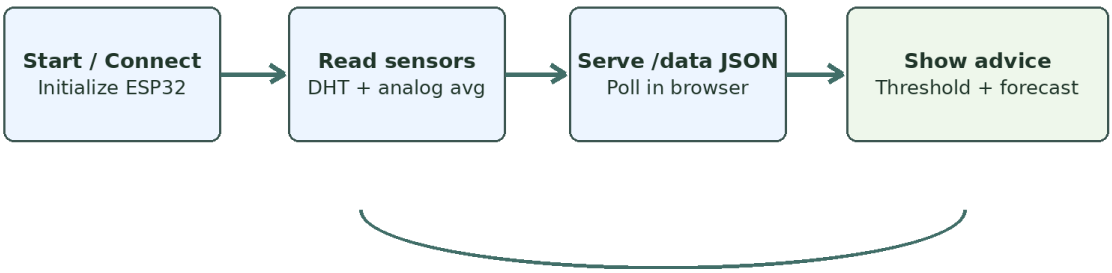


Figure 3.3: Flowchart of sensing and decision-support process

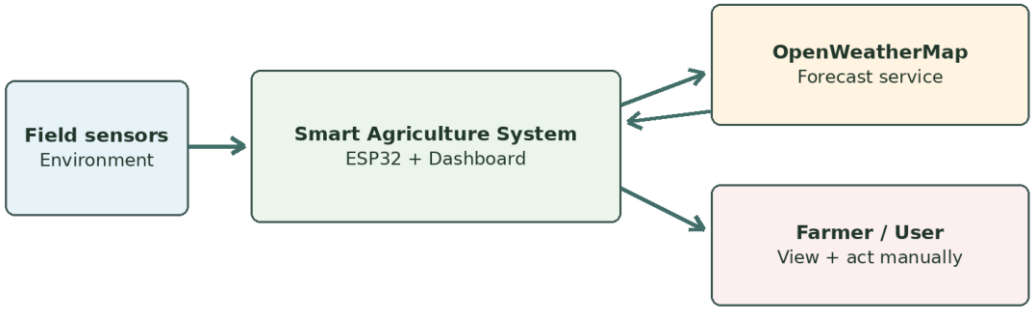


Figure 3.4: Data Flow Diagram – Level 0

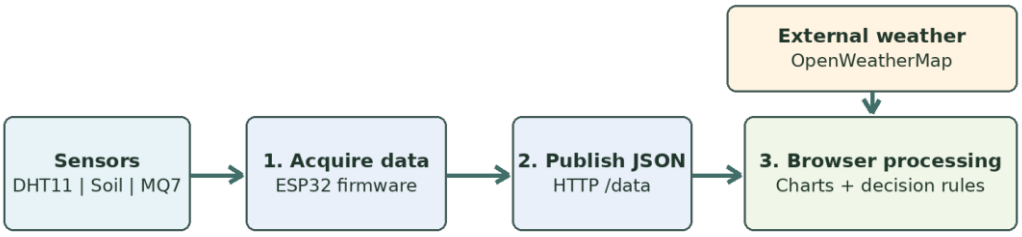


Figure 3.5: Data Flow Diagram – Level 1

4 Implementation

4.1 Implementation Plan for Sem – 8

The present Semester VII prototype is functional for live sensor monitoring and advisory recommendations. The proposed Semester VIII work below is adapted from the future-direction section of the existing Agriculture report. It focuses on measurement quality, reliability, records and safe access, rather than automatic actuation.

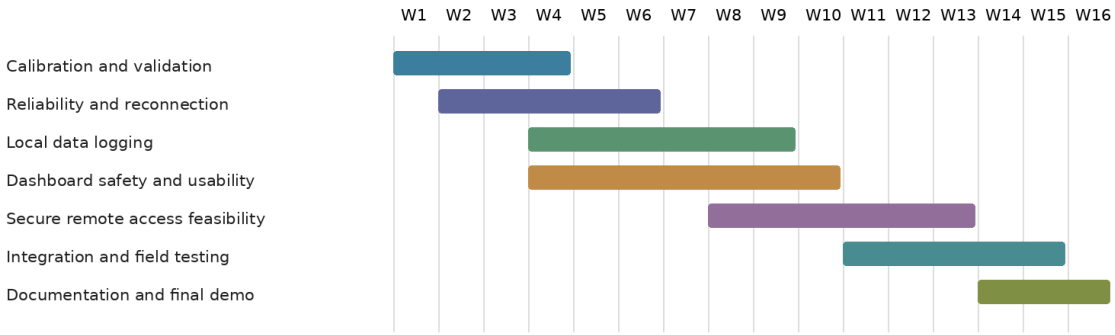


Figure 4.1: Proposed Semester VIII implementation schedule

Table 4.1: Work Breakdown Structure for Sem – 8

WBS	Task	Weeks	Deliverable
1.0	Documented soil and sensor calibration	W1–W4	Calibration procedure and test notes
2.0	ESP32 reconnection and failure handling	W2–W6	More reliable status reporting
3.0	Historical logging and export	W4–W9	Locally retained or secure stored readings
4.0	Dashboard validation and usability	W4–W10	Safer input handling, clear stale-data indication
5.0	Secured access feasibility	W8–W13	Secure design proposal / prototype
6.0	Integration and environmental testing	W11–W15	Functional evaluation record
7.0	Documentation and final demonstration	W14–W16	Final report and live demo

Table 4.2: Individual contribution

Roll No.	Name	Contribution (%)
07	Harshal Bhosale	25%
13	Rakesh Chaudhary	25%
15	Pranit Chiman	25%
73	Om Darekar	25%

4.2 Coding Standard

Naming conventions used in the supplied Arduino firmware and browser dashboard are summarized below. The full Arduino listing and representative dashboard code appear in Appendix III.

Table 4.3: Naming conventions used

Item	Convention	Examples
Arduino compile-time constants	Uppercase snake case / defines	WIFI_SSID, SOIL_PIN, MQ7_PIN
Arduino functions	lower camel case	readSensors, handleData
Arduino state	lower camel case	dhtOk, gasRaw, lastRead
Dashboard functions	lower camel case	soilPct, evaluate, fetchWeather
DOM element IDs	short or kebab-case	ip, t-temperature, v-soilPct
Settings fields	short camel case	soilLo, soilHi, tW, gC

The sketch separates Wi-Fi configuration, sensor sampling, HTTP headers/handlers, setup and the main loop. The dashboard separates styling, markup, storage, calibration, visualization, recommendation evaluation and network access. Error feedback is displayed in the interface, and the ESP32 prints its readings to the Serial Monitor.

4.3 Testing

The functioning prototype was tested by its project team on the real ESP32 and connected sensors. The team confirmed correct display of the four sensor outputs, local dashboard access and successful weather forecast loading. Code review additionally establishes the expected functions of the data endpoint and threshold rules. Formal numerical accuracy and latency measurements have not been supplied.

4.3.1 Test Cases

Table 4.4: Functional test cases

TC	Module	Test description	Expected result
1	ESP32	Power the sensor node	Firmware starts and joins Wi-Fi
2	DHT11	Read temperature and humidity	Numeric values in JSON
3	Soil probe	Read analog moisture channel	Raw ADC data shown as estimated %
4	MQ7	Read gas channel	Raw gas value displayed
5	HTTP	Request ESP32 /data	Valid JSON with five fields
6	Dashboard	Connect using local IP	Connection status and values update
7	Calibration	Edit dry/wet settings	Estimated soil percentage changes
8	Rule engine	Simulate dry soil below limit	Irrigation recommendation appears
9	Forecast	Supply city and valid API key	Near-term forecast loads
10	Rain-aware rule	Dry soil and rain probability >= 60%	Advice may postpone irrigation
11	Alerts	Raise temperature/gas input	Corresponding configured warning
12	Fault display	DHT read fails	dhtOk=false available for warning

4.3.2 Results of Testing and System Performance

Table 4.5: Results of functional testing

Test area	Observed / supported outcome	Evidence
Sensor hardware	All four readings display normally	User-confirmed working hardware
Local dashboard	Locally opened dashboard connects successfully	User-confirmed operational demo
Weather forecast	OpenWeatherMap forecast loads	User-confirmed successful API use
JSON and rules	Fields and rule paths present in source	Source code review
Quantitative accuracy	No calibrated measurement dataset provided	Not assessed
Network latency	No benchmark measurements provided	Not assessed

Integration result: the actual demonstration confirms the useful end-to-end chain from three sensors to ESP32 and local browser and from the external forecast API to the dashboard. As with any prototype, uncalibrated gas output must not be interpreted as a certified gas-safety reading. No claim of automatic irrigation, water savings, crop-yield improvement or sub-second latency is made.

The confirmed operating setup consists of the ESP32 and three attached modules, each powered at 3.3 V according to the project team, together with a locally opened dashboard. Temperature, humidity, estimated soil moisture and raw MQ7 values were visible in the user interface. The weather panel also returned forecast information when configured with a valid city and API key. These observations support the working-prototype status reported here.

The listed test cases describe both demonstrated capabilities and logical paths that can be examined in the supplied source. They are not presented as a set of instrument-calibrated trials. In particular, the real hardware demonstration does not establish sensor precision, certified gas thresholds, long-term Wi-Fi reliability or quantified agricultural benefits. The future-work plan therefore reserves dedicated activities for calibration, failure handling, safe access and broader validation.

5 Results and Discussion

The Smart Agriculture Monitoring System was demonstrated as a local-network sensor-to-browser application, with external weather information shown in the same dashboard. The images below are taken from the user's own project report.

Intermediate results

- ESP32 firmware reads digital DHT11 and analog soil/MQ7 channels and publishes cached values through /data.
- The dashboard processes the payload and displays live values, status and small trend graphs.
- OpenWeatherMap is configured separately in the browser and supplies the next 12 hours of forecast information.

Final results: hardware integration

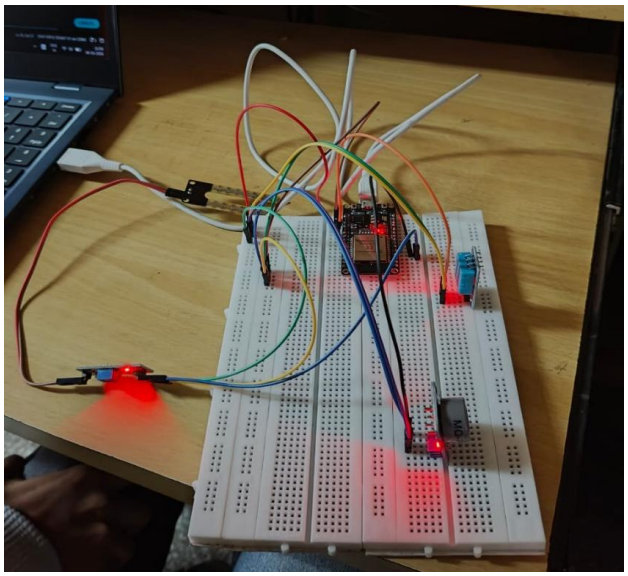


Figure 5.1: Complete Smart Agriculture hardware setup

The breadboard-based prototype connects the ESP32 to all three sensor modules. The project team confirmed correct hardware operation at 3.3 V and correct readings on the local dashboard.

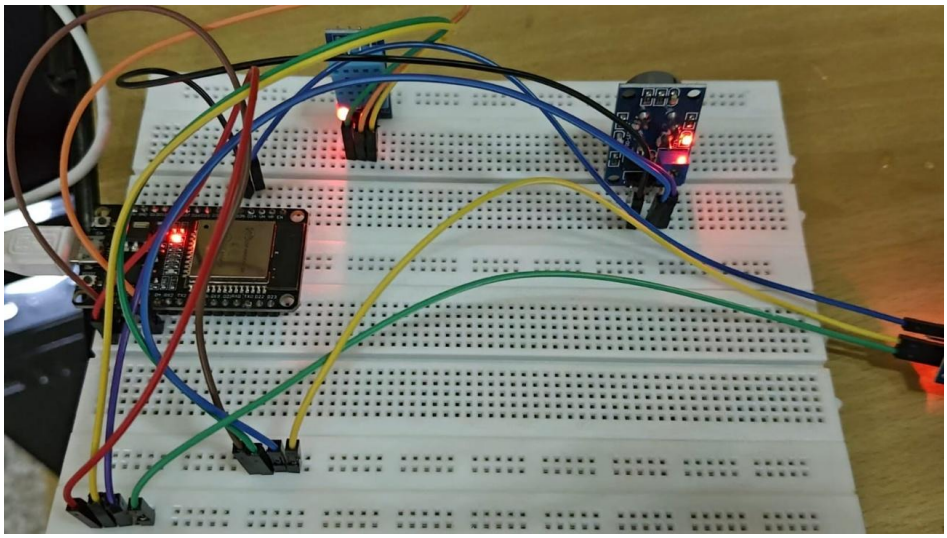


Figure 5.2: ESP32 and sensor connection close-up

The close-up demonstrates direct pin-level interfacing of the sensor modules and shared ground connections. The setup is for academic demonstration; environmental hardening remains an identified improvement.

Final results: monitoring and decision support

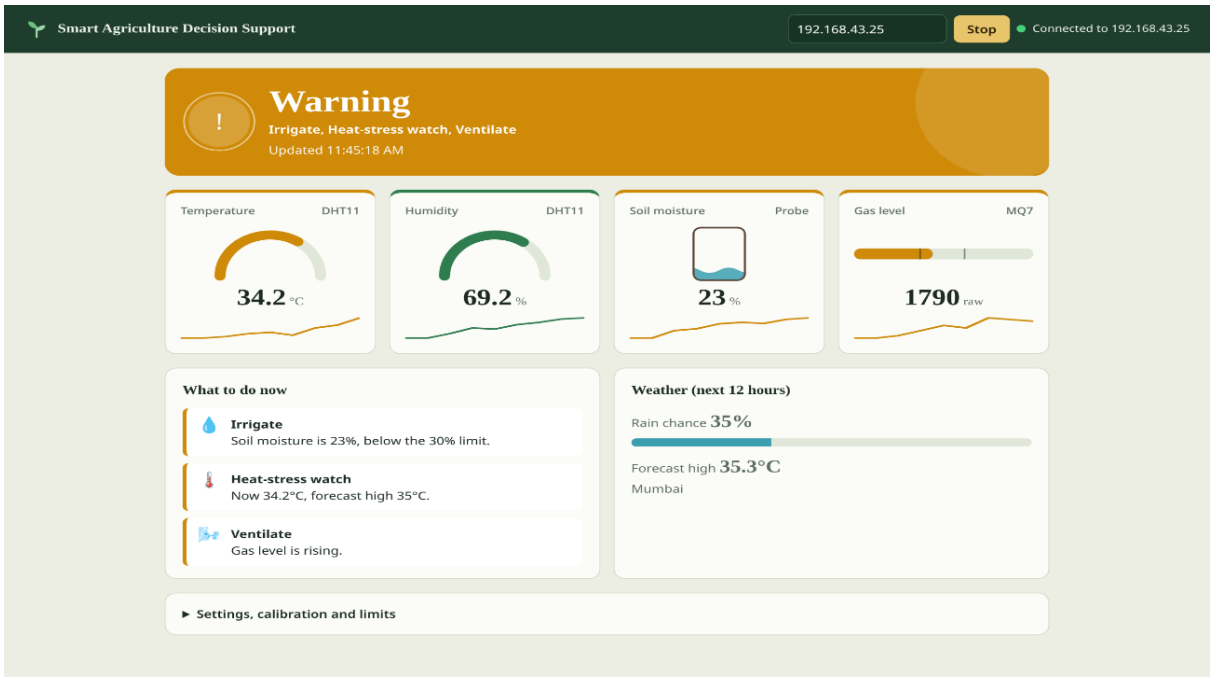


Figure 5.3: Web dashboard with active warnings

The dashboard presents temperature, humidity, moisture and raw gas indicators, plus rule-generated warnings and weather information. These views make the threshold logic visible to the user without operating hardware actuators.

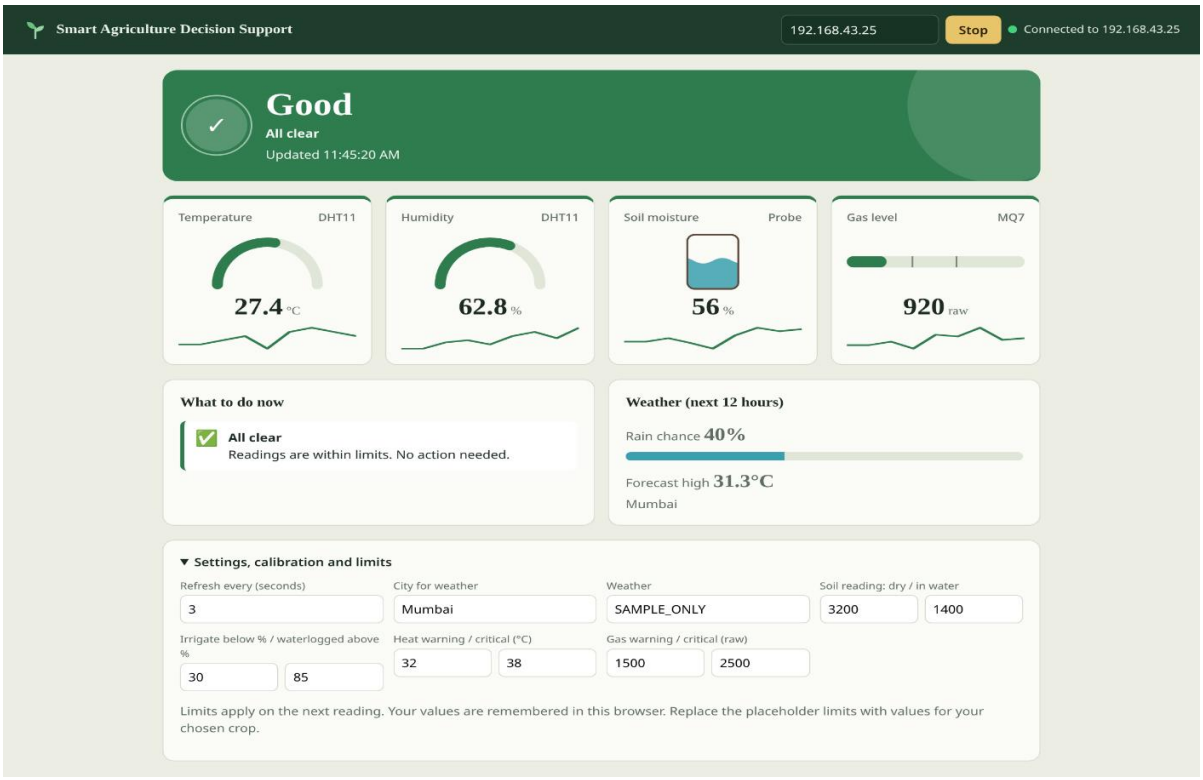


Figure 5.4: Web dashboard and calibration settings

The configuration panel allows changes to soil calibration, warning thresholds, city and weather key. Browser localStorage retains these settings; the mini-history graphs contain the last 30 readings in memory, not a persistent historical database.

Comparison with existing practice

Table 5.1: Comparison between proposed system and conventional practice

Criterion	Traditional inspection	Smart Agriculture system
Measurement access	Occasional manual checks	Live local browser readings
Soil advisory	Depends on field judgement	Moisture estimate and editable limits
Rain awareness	Manual consultation	Integrated 12-hour forecast
Heat / gas indicators	Not necessarily available	Visible warnings with caveats
Autonomous irrigation	No	No: user-directed actions
Stored history	Manual notes	Temporary 30-sample browser trends

Discussion: the project illustrates how locally measured information and external forecast data can be combined to support agricultural decisions. It does not replace agronomic judgement, laboratory calibration or certified environmental safety instrumentation.

6 Conclusion & Future Work

Conclusion

The IoT Based Smart Agriculture Monitoring System has been designed and demonstrated as a working ESP32 monitoring prototype. It collects temperature, humidity, soil and raw MQ7 sensor values, provides them through a lightweight local HTTP endpoint and displays them using a custom browser dashboard.

The dashboard combines live readings, sensor status, trends, editable limits and OpenWeatherMap forecast information to support decisions about irrigation, waterlogging and environmental conditions. In contrast with the previous semester's Blynk-based approach, the current solution uses an independent HTML/CSS/JavaScript interface and a local ESP32 data service.

The work brings together IOE Things (sensor modules and ESP32), Data (JSON readings and forecast values), Process (calibration and rule evaluation), and People (the farmer interpreting recommendations). The final project is explicitly monitoring and recommendations only. It does not switch a pump, fan, relay or other actuator.

Future Work

1. Better sensor characterization: document soil wet/dry calibration; evaluate a capacitive soil probe and a more accurate ambient sensor; preserve the distinction between raw MQ7 output and calibrated concentrations.

2. Local history and analysis: store readings for longer-term trend views and enable export, instead of retaining only short in-browser trend arrays.

3. Connection and safety feedback: show stale-reading status, handle Wi-Fi interruption and DHT failures more robustly, and validate configuration fields.

4. Secure access and credentials: evaluate authenticated access, secure transmission and safer storage of OpenWeatherMap credentials before wider deployment.

5. Improved environmental readiness: protect the enclosure and wiring, test power stability and consider maintainability during extended operation.

6. Better decision support: evaluate crop-specific limits and the reliability of weather-informed recommendations using documented observations.

Learning from Project Management

- Keeping ESP32 acquisition and browser decision logic separate simplifies revisions and testing.
- Testing on the real hardware and checking the serial output is important even when dashboards render correctly.
- Actual measured outcomes should be distinguished from source-code capabilities and proposed enhancements.
- Future improvements should be planned with safety, sensor calibration and credential protection in mind.

Appendix - I

Data Sheet(s) - Electronic Component

Table A.1: ESP32 development board

Parameter	Specification
Role	Sensor acquisition and HTTP/Wi-Fi service
Processor	Dual-core Xtensa LX6, up to 240 MHz
Wi-Fi	Integrated 2.4 GHz 802.11 b/g/n
ADC	12-bit analog inputs; raw range 0–4095
GPIO logic	3.3 V
Connected analog inputs	GPIO34 and GPIO35 (ADC1)

Table A.2: DHT11 sensor

Parameter	Specification
Type	Digital ambient temperature and humidity sensor
Temperature	0–50°C; typical accuracy ±2°C
Humidity	20–90% RH; typical accuracy ±5%
Interface	Single-wire digital output
Power	3.3 V on this prototype
ESP32 connection	GPIO4

Table A.3: Soil moisture sensor

Parameter	Specification
Type	Resistive analog soil probe
Readout	Raw ADC converted to indicative % in browser
Calibration	Editable defaults: dry 3200; wet 1400
Thresholds	Below 30% dry; above 85% possible waterlogging
Power	3.3 V on this prototype
ESP32 connection	GPIO34

Table A.4: MQ7 module

Parameter	Specification
Function	Raw semiconductor gas response
Readout	Analog ADC value (not CO ppm)
Warning settings	1500 raw warning; 2500 raw critical
Power	3.3 V in user-tested prototype; module-specific heater requirements must be verified
ESP32 connection	GPIO35
Qualification	No documented heater cycling or certified CO calibration

Installation Procedure - Development Software

1. Arduino IDE: install Arduino IDE and the ESP32 board package from Espressif using Boards Manager; choose the connected ESP32 development board and port.

2. DHT library: install the Adafruit DHT sensor library and any required dependencies using the Arduino Library Manager.

3. Firmware: open esp32_web_server.ino; configure the local Wi-Fi SSID and password privately; check pin assignments and upload the sketch.

4. Serial diagnostics: open Serial Monitor at 115200 baud; verify sensor output and record the IP address assigned to the ESP32.

5. HTTP check: on a browser connected to the same reachable network, request http://ESP32-IP/data and verify the JSON response.

6. Dashboard: open smart_agri_dashboard_v2.html locally, enter the ESP32 IP in the top bar and select Connect.

7. Weather: enter the city and an OpenWeatherMap key in the settings panel; verify that the near-term forecast loads.

8. Calibration: replace the example wet/dry values and warning limits as appropriate to the actual sensors and growing conditions. No automatic actuator setup is required.

Appendix - II

Software Requirement Specification for the IoE Smart Agriculture Monitoring System

1 Introduction

1.1 Purpose. This SRS describes functional and nonfunctional requirements for the local ESP32 sensor-monitoring and browser-based advisory system.

1.2 Scope. The system acquires environmental measurements, publishes them to an HTTP/JSON service, displays live values, incorporates external weather forecasts and provides rule-based suggestions. It performs no automatic actuation.

1.3 Definitions. IOE – Internet of Everything; ADC – Analog-to-Digital Converter; HTTP – Hypertext Transfer Protocol; JSON – JavaScript Object Notation; RH – Relative Humidity; API – Application Programming Interface.

1.4 Reference documents. ESP32 technical manual, DHT11 data sheet, MQ7 manufacturer documentation, OpenWeatherMap API documentation and the supplied source code.

2 Overall Description

2.1 Product perspective. A local Wi-Fi sensing node and standalone browser interface, plus a browser-accessed internet weather service.

2.2 Product functions. Sensor readings, JSON data, live gauges and trends, editable calibration/limits, connection indicator, forecast-aware advice and status warnings.

2.3 User characteristics. Farmers and project evaluators with access to an ordinary web browser and basic familiarity with local-network connectivity.

2.4 Constraints. The browser must reach the ESP32 IP; the forecast requires internet access and a working API key; the resistive probe and MQ7 require appropriate calibration and interpretation.

2.5 Assumptions. The sensor node has stable power, all connected modules share a common ground, and the user has a local network available.

3 Specific Requirements

3.1 Functional requirements

FR-1. The ESP32 shall read temperature and humidity from the DHT11 on GPIO4.

FR-2. The ESP32 shall sample soil and gas analog inputs from GPIO34 and GPIO35 and publish averaged raw readings.

FR-3. The ESP32 shall expose the latest readings in JSON at /data, including dhtOk status.

FR-4. The dashboard shall display live values, connection status and short trend information.

FR-5. The dashboard shall compute indicative moisture percentage using editable dry/wet ADC values.

FR-6. The dashboard shall show advisory messages based on configurable soil, heat, humidity and raw gas limits.

FR-7. When weather is configured, the dashboard shall consider rain probability before recommending irrigation.

FR-8. The system shall not actuate irrigation pumps, fans or other field equipment.

3.2 External interfaces. DHT11 digital signal on GPIO4; analog sensors on GPIO34 and GPIO35; ESP32 Wi-Fi/HTTP service; HTML/CSS/JavaScript dashboard; HTTPS request to OpenWeatherMap.

3.3 Performance and quality requirements. The Arduino sketch refreshes cached sensor values about every two seconds; the browser polls every three seconds by default. The interface should accurately distinguish stale or unsuccessful sensor data from live reliable measurements. No guaranteed latency or calibrated accuracy has been established from the supplied materials.

Appendix - III

A. ESP32 Sensor Node Firmware (Arduino C++)

The following listing reproduces the project's firmware logic, with Wi-Fi credentials and unused API credential values replaced by placeholders for safe circulation. The operational source is esp32_web_server.ino.

```
/*
  Smart Agriculture DSS - ESP32 web server (no MQTT)
  The ESP32 reads DHT11 + soil + MQ7 and serves them as JSON at:
      http://<ESP32-IP>/data
  The dashboard page fetches that address every few seconds.

  Libraries (Library Manager): DHT sensor library (Adafruit), Adafruit Unified Sensor
  (WiFi and WebServer come with the ESP32 board package)
*/

#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"

// ---------- EDIT THESE ----------
const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

const char* WEATHER_API_KEY = "YOUR_API_KEY";
const char* WEATHER_CITY = "Mumbai";
// --------------------------------

#define DHTPIN   4
#define DHTTYPE  DHT11
#define SOIL_PIN 34   // ADC1, input only
#define MQ7_PIN  35   // ADC1, input only (use a voltage divider if the module runs at 5 V)

DHT dht(DHTPIN, DHTTYPE);
WebServer server(80);

float temperature = 0, humidity = 0;
int soilRaw = 0, gasRaw = 0;
bool dhtOk = false;
unsigned long lastRead = 0;

int readAnalogAvg(int pin) {
  long sum = 0;
  for (int i = 0; i < 10; i++) { sum += analogRead(pin); delay(5); }
  return sum / 10;
}

void readSensors() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  dhtOk = !(isnan(h) || isnan(t));
  if (dhtOk) { humidity = h; temperature = t; }
  else Serial.println("DHT11 read failed - check wiring");
  soilRaw = readAnalogAvg(SOIL_PIN);   // higher = drier
  gasRaw  = readAnalogAvg(MQ7_PIN);    // higher = more gas
}

void sendCORS() {
  server.sendHeader("Access-Control-Allow-Origin", "*");   // lets the dashboard page read
this data
  server.sendHeader("Cache-Control", "no-store");
}

void handleData() {
  sendCORS();
  char json[160];
  snprintf(json, sizeof(json),
           "{\"temperature\":%.1f,\"humidity\":%.1f,\"soil\":%d,\"gas\":%d,\"dhtOk\":%s}",
           temperature, humidity, soilRaw, gasRaw, dhtOk ? "true" : "false");
  server.send(200, "application/json", json);
}

void handleRoot() {
  sendCORS();
  server.send(200, "text/plain", "ESP32 smart agriculture node is running. Data at /data");
```

```
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(SOIL_PIN, INPUT);
  pinMode(MQ7_PIN, INPUT);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println();
  Serial.print("Connected. Open this in the dashboard: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/data", handleData);
  server.begin();
  readSensors();
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {      // simple auto-reconnect
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    delay(3000);
  }
  server.handleClient();

  if (millis() - lastRead >= 2000) {        // DHT11 needs at least 1 s between reads
    lastRead = millis();
    readSensors();
    Serial.printf("T=%.1f C  H=%.1f %%  Soil=%d  Gas=%d\n", temperature, humidity, soilRaw,
gasRaw);
  }
}
```

B. Browser Data Processing (JavaScript excerpts)

The complete dashboard is supplied as smart_agri_dashboard_v2.html. The representative code below illustrates calibration, decision rules, polling and weather access without duplicating all visual styles in the printed appendix.

Function soilPct()

```
function soilPct(raw){                          // resistive probe: higher reading = drier
  const dry = num('dry'), wet = num('wet');
  return Math.max(0, Math.min(100, (dry - raw) / (dry - wet) * 100));
}
```

Function evaluate()

```
function evaluate(d){
  const out = [], add = (level, ic, title, msg) => out.push({level, ic, title, msg});
  const rainSoon = rain && rain.pop >= 0.6;
  if (d.soilPct < num('soilLo')) {
    if (rainSoon) add('good', '🌧', 'Suppress irrigation', `Soil is dry
(${d.soilPct.toFixed(0)}%) but rain is likely (${Math.round(rain.pop * 100)}% chance).
Wait.`);
    else add('warning', '💧', 'Irrigate', `Soil moisture is ${d.soilPct.toFixed(0)}%, below
the ${num('soilLo')}% limit.`);
  } else if (d.soilPct > num('soilHi')) add('warning', '🚫', 'Hold irrigation', `Soil
moisture is ${d.soilPct.toFixed(0)}%. Waterlogging risk.`);

  if (d.temperature >= num('tC')) add('critical', '🔥', 'Heat stress', `${d.temperature}°C
is at or above the critical limit.`);
  else if (d.temperature >= num('tW') || (rain && rain.tmax >= num('tC'))) add('warning',
'🌡', 'Heat-stress watch', `Now ${d.temperature}°C${rain ? `, forecast high
${rain.tmax.toFixed(0)}°C` : ''}.`);

  if (d.gas >= num('gC')) add('critical', '☠', 'Ventilate now', 'Gas level is critical.
Open vents and keep people out of enclosed areas.');
  else if (d.gas >= num('gW')) add('warning', '💨', 'Ventilate', 'Gas level is rising.');
  else if (d.humidity > 85 && d.temperature > 30) add('warning', '💨', 'Ventilate', 'High
heat and humidity. Disease risk.');

  if (!out.length) add('good', '✅', 'All clear', 'Readings are within limits. No action
needed.');
```

```
  return out;
}
```

Function fetchWeather()

```
function fetchWeather(){
  if (!$('key').value || !$('city').value) return;
  try {
    const r = await
fetch(`https://api.openweathermap.org/data/2.5/forecast?q=${encodeURIComponent($('city').val
ue)}&units=metric&cnt=4&appid=${$('key').value}`);
    if (!r.ok) throw new Error(r.status);
    const j = await r.json();
    rain = {pop: Math.max(...j.list.map(x => x.pop || 0)), tmax: Math.max(...j.list.map(x =>
x.main.temp_max))};
    $('wx').innerHTML = `<div>Rain chance <span class="num">${Math.round(rain.pop *
100)}%</span></div>
      <div class="rainbar"><i id="rb"></i></div>
      <div>Forecast high <span class="num">${rain.tmax.toFixed(1)}°C</span></div>
      <div class="muted">${j.city ? j.city.name : ''}</div>`;
    requestAnimationFrame(() => { const b = $('rb'); if (b) b.style.width = rain.pop * 100 +
'%'; });
    if (last) render(last);
  } catch (e) { $('wx').textContent = 'Weather unavailable (' + e.message + '). Check the
city name and API key.'; }
}
```

References

[1] Espressif Systems, “ESP32 Technical Reference Manual.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] DHT11 Temperature and Humidity Sensor, “Technical Data Sheet.” [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[3] Winsen Electronics, “MQ-7 Carbon Monoxide Sensor Datasheet.” [Online]. Available: https://www.winsen-sensor.com/

[4] Adafruit Industries, “DHT Sensor Library for Arduino.” [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[5] OpenWeather, “5 Day / 3 Hour Forecast API.” [Online]. Available: https://openweathermap.org/forecast5

[6] Espressif Systems, “Arduino ESP32 WiFi Library Documentation.” [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[7] Espressif Systems, “Arduino ESP32 WebServer Library.” [Online]. Available: https://github.com/espressif/arduino-esp32/tree/master/libraries/WebServer

[8] A. Tzounis et al., “Internet of Things in Agriculture: Recent Advances and Future Challenges,” Biosystems Engineering, vol. 164, pp. 31–48, 2017.

[9] Arduino, “Arduino Documentation.” [Online]. Available: https://docs.arduino.cc/

[10] Espressif Systems, “Arduino Core for the ESP32.” [Online]. Available: https://github.com/espressif/arduino-esp32

Acknowledgements

We express our sincere gratitude to our project guide, Prof. Janhavi Baikerikar, for guidance and constructive feedback throughout the development of the IoT Based Smart Agriculture Monitoring System. Her support helped us refine the embedded-sensing work and its custom web dashboard.

We also thank the Department of Information Technology, Don Bosco Institute of Technology, Mumbai, for providing an environment in which the hardware and software aspects of this Semester VII project could be developed and demonstrated.

Finally, we are grateful to our families, classmates and peers for their encouragement during prototype development, testing and documentation.

(_____)
(Harshal Bhosale - 07)

(_____)
(Rakesh Chaudhary - 13)

(_____)
(Pranit Chiman - 15)

(_____)
(Om Darekar - 73)

Date:

SDG

SDG 2 – Zero Hunger: Monitoring supports timely crop-management choices.

SDG 6 – Clean Water and Sanitation: Rain-aware moisture advice can discourage unnecessary irrigation.

SDG 9 – Industry, Innovation and Infrastructure: Low-cost sensors, Wi-Fi and a custom interface demonstrate connected infrastructure.

SDG 12 – Responsible Consumption and Production: Condition-based suggestions support responsible water use.

SDG 13 – Climate Action: Forecast information aids responses to changing environmental conditions.

These alignments describe intended relevance. The present project does not claim experimentally measured crop-yield, water-saving or climate outcomes.

IoE-Based Smart Agriculture Monitoring with ESP32, Local Web Visualization and Weather-Aware Decision Support

Harshal Bhosale, Rakesh Chaudhary, Pranit Chiman and Om Darekar

Department of Information Technology, Don Bosco Institute of Technology, Mumbai, India

Abstract— This paper presents an Internet of Everything (IOE) based Smart Agriculture Monitoring System using an ESP32, DHT11, resistive soil moisture probe and MQ7 module. The ESP32 exposes readings through a local HTTP/JSON endpoint; a standalone HTML/CSS/JavaScript dashboard displays live values and short trends, computes estimated soil moisture, and offers configurable environmental advisories. OpenWeatherMap forecast information can modify irrigation suggestions when rainfall is likely. The project has been demonstrated on working hardware, and does not automatically control any actuator.

Index Terms— Internet of Everything, smart agriculture, ESP32, soil moisture, HTTP, decision support, OpenWeatherMap.

I. INTRODUCTION

Agricultural decisions require timely information about soil and microclimatic conditions. Manual inspection can miss rapid changes. An IOE-oriented approach combines sensing devices, data, rules and the person taking action. This prototype provides local readings and advisory recommendations without automatic irrigation.

II. RELATED WORK

The ESP32 technical reference [1] supports the selection of an integrated Wi-Fi and ADC platform. The DHT11 data sheet [2] establishes the sensor's basic ambient-measurement limitations. MQ7 documentation [3] motivates the decision to treat its signal as a raw analog value rather than a calibrated CO result. The OpenWeather API [5] supplies near-term forecast information; the cited agriculture IoT review [8] provides broader context.

III. SYSTEM ARCHITECTURE

The sensing layer contains DHT11 (GPIO4), a resistive soil probe (GPIO34) and MQ7 (GPIO35). Arduino firmware averages ten analog samples per channel, refreshes values approximately every two seconds, and exposes JSON fields temperature, humidity, soil, gas and dhtOk at HTTP /data. A browser dashboard polls the ESP32 over accessible local Wi-Fi.

Independently, the browser requests four forecast samples from OpenWeatherMap. The browser preserves user configuration in localStorage and renders animated gauges, mini-trends, warnings and recommended actions. The architecture contains no Blynk-specific cloud service and no database that permanently stores measurements.

IV. METHODS

Soil moisture is estimated by M = clamp((D - R)/(D - W) × 100, 0, 100), where D and W are dry/wet calibration readings and R is the raw ADC result. With defaults D=3200 and W=1400, a raw value of 2700 gives M≈27.8%. Below the configured 30% low threshold, irrigation is suggested unless forecast rain probability is at least 60%. A value above 85% flags possible waterlogging.

Other rules compare ambient temperature against the configurable 32°C/38°C warning levels, and raw gas ADC against 1500/2500 values. These settings are demonstration parameters, not crop-specific agronomic limits or certified gas-concentration thresholds. The DHT failure flag allows error awareness.

V. IMPLEMENTATION AND RESULTS

The ESP32 was operated with the three attached sensors, and the project team confirmed that four displayed measurements are normal in the local dashboard. The OpenWeatherMap forecast also loaded successfully. The interface therefore demonstrates the integrated sensor-to-browser and forecast-to-advice flow. No field yield, water-saving, gas-calibration accuracy or response-time benchmark is reported.

VI. DISCUSSION

The local-first design reduces reliance on a proprietary dashboard account. However, it depends on reachable Wi-Fi, has limited in-memory trend history, and uses browser-stored weather credentials. The user remains responsible for the final irrigation or ventilation decision. More rigorous sensor calibration and failure-state handling would improve practical reliability.

VII. CONCLUSION

The completed prototype combines inexpensive sensing, ESP32 HTTP data access, independent browser visualization and forecast-aware advisory rules. It demonstrates the four pillars of IOE while keeping all actuation manual. Future work can improve secure access, historical retention, calibration and environmental robustness.

REFERENCES

[1] Espressif Systems, "ESP32 Technical Reference Manual." [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] DHT11 Temperature and Humidity Sensor, "Technical Data Sheet." [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[3] Winsen Electronics, "MQ-7 Carbon Monoxide Sensor Datasheet." [Online]. Available: https://www.winsen-sensor.com/

[4] Adafruit Industries, "DHT Sensor Library for Arduino." [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[5] OpenWeather, "5 Day / 3 Hour Forecast API." [Online]. Available: https://openweathermap.org/forecast5

[6] Espressif Systems, "Arduino ESP32 WiFi Library Documentation." [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[7] Espressif Systems, "Arduino ESP32 WebServer Library." [Online]. Available: https://github.com/espressif/arduino-esp32/tree/master/libraries/WebServer

[8] A. Tzounis et al., "Internet of Things in Agriculture: Recent Advances and Future Challenges," Biosystems Engineering, vol. 164, pp. 31–48, 2017.